物件導向程式設計 2

A1143305 吳宥穎

U1_0 登入視窗

AI 對話紀錄



請幫我生成一個程式並附上每行註解

這是一份根據圖片中的 Java Swing 登入視窗程式碼進行重新編寫並**加上每行詳細中文註解**的完整程式碼：

Java

```
import javax.swing.*; // 導入 Java Swing 圖形介面套件
import java.awt.*;    // 導入 Java AWT 圖形介面相關工具套件
```

```
: javax.swing.*; // 導入 Java Swing 圖形介面套件
: java.awt.*;    // 導入 Java AWT 圖形介面相關工具套件

: class BadLogin extends JFrame { // 定義一個名為 BadLogin 的類別，繼承自 JFra
ıblic BadLogin() { // 建構子，用來初始化視窗元件與屬性
   setTitle("登入"); // 設定視窗標題為「登入」
   setSize(300, 200); // 設定視窗大小為寬 300 像素、高 200 像素
   setVisible(true); // 設定視窗為可見狀態
   setLayout(null); // 取消預設的佈局管理器，改為手動絕對座標定位

   JLabel l1 = new JLabel("帳號:"); // 建立一個顯示文字「帳號:」的標籤物件
   JTextField t1 = new JTextField(); // 建立一個供使用者輸入帳號的文字框物件
   JLabel l2 = new JLabel("密碼:"); // 建立一個顯示文字「密碼:」的標籤物件
   JTextField t2 = new JTextField(); // 建立一個供使用者輸入密碼的文字框物件（實
   JButton btn = new JButton("登入"); // 建立一個顯示文字「登入」的按鈕物件

   add(l1); add(t1); add(l2); add(t2); add(btn); // 將上述建立的標籤、文字框與
```

你可以幫我看一下這段程式碼，雖然是完整的，但是有沒有什麼問題嗎？

這段程式碼雖然可以正常編譯並執行，但裡面其實藏了幾個 Java Swing 開發中常見的**經典「Bug」與設計盲點**：

1. 字串比較錯誤（Bug）

- **問題**：`if (t1.getText() == "admin" && t2.getText() == "")`
- **原因**：在 Java 中，`==` 是用來比較兩個物件的**記憶體位址**（是否為同一個物件），而不是比較字串的**內容**。因此這段條件幾乎永遠會判定為 `false`，導致你輸入正確的帳密時，主控台也不會印出「登入成功」。
- **正確做法**：字串內容比對必須使用 `.equals()` 方法：

```java
if (t1.getText().equals("admin") && t2.getText().equals(""))
```

2. 沒有設定元件的座標與大小（介面排版問題）

- **問題**：程式碼中使用了 `setLayout(null);`（取消預設佈局），但**完全沒有**對任何元件設定 `setBounds(x, y, width, height)`。
- **原因**：在 `null` 絕對座標佈局下，所有未設定位置與大小的元件預設寬高皆為 0，且全部會堆疊在視窗的左上角，導致你打開視窗後會**什麼都看不到**（一片空白）。
- **正確做法**：
  - 要嘛幫每個元件加上座標，例如：`l1.setBounds(30, 30, 50, 20);`、`t1.setBounds(90, 30, 120, 20);` … 等等。
  - 要嘛改用內建的佈局管理器（如 `FlowLayout` 或 `GridLayout`），讓系統自動幫你排列元件。

3. 密碼欄位用了明碼輸入（安全性與體驗問題）

- **問題：** `JTextField t2 = new JTextField();`
- **原因：** `JTextField` 是普通的文字輸入框，使用者輸入密碼時，畫面會直接顯示出明文。
- **正確做法：** 密碼欄位應該使用 `JPasswordField`，它會自動將輸入的字元遮罩（顯示成點點或星號）。

4. 視窗關閉行為未定義

- **問題：** 缺少 `setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);`。
- **原因：** 如果沒有加上這行，當你點擊視窗的「X」關閉按鈕時，視窗雖然會消失，但 Java 程式其實還在背景默默運行（行程未結束）。
- **正確做法：** 在建構子中加上：

```java
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
```

5. `setVisible(true)` 的呼叫時機

- **問題：** `setVisible(true)` 被寫在建構子的前半段。
- **原因：** 雖然這樣寫視窗通常還是打得開，但標準的 Swing 開發規範建議將 `setVisible(true)` 放在**所有元件都新增並設定完畢的最後一行**，避免視窗在元件還沒載入完時就提前顯示，造成畫面閃爍或元件渲染不完全。

Reflection

AI 給的哪裡有問題

1. 原程式碼使用 == 進行帳號與密碼的比對。在 Java 中，== 是比較兩物件之記憶體位址而非字串內容，導致比對結果恆為 false，無法順利完成驗證。
2. 程式碼宣告了 setLayout(null);（採用絕對定位），但未呼叫 setBounds() 設定各元件（標籤、文字框、按鈕）的座標與尺寸，因此所有元件大小為 0 且堆疊於左上角，畫面呈現空白。
3. 密碼輸入框採用普通單行文字框 JTextField，輸入時會以明碼顯示，不符合隱私。
4. 未設定 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);。當使用者關閉視窗時，僅隱藏介面，後台 Java 行程並未終止，會持續佔用系統資源。

5. 於建構子前半段即呼叫 setVisible(true)，此時介面元件尚未完全載入與佈局，容易導致畫面呈現時產生閃爍或繪製不全。

改了甚麼? 為甚麼改?

1. 修正字串比對語法

修改：將條件判斷改為 t1.getText().equals("admin")。

修正原因：改用 .equals() 方法比對字串內容，確保帳號密碼驗證邏輯正常運作。

2. 補充元件座標設定或改佈局

修改：針對各元件補設定 setBounds(x, y, width, height)，或改採用 FlowLayout / GridLayout 等佈局管理器。

修正原因：賦予元件明確的位置與大小，確保介面元件能於視窗內正確渲染與顯示。

3. 更換密碼欄位元件

修改：將密碼輸入框由 JTextField 替換為 JPasswordField。

修正原因：使輸入之密碼能自動轉換為遮罩字元（如 ・・・・），確保敏感資訊不外洩。

4. 加入視窗關閉機制

修改：加入 setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);

修正原因：確保使用者關閉視窗時能同步結束程式行程

5. 調整視窗顯示順序

修改：將 setVisible(true) 移至建構子末端（所有元件完成 add() 之後）

修正原因：確保所有 GUI 元件加載與配置完成後才一次性呈現視窗，提高畫面渲染之穩定度。